

Trummer Heterogeneous Joins

Semantic join variants, learned cascades, threshold sweeps, and next steps

Cheap-only block join → expensive-only block join → row-wise cascade → batch-wise cascade

Master 2 project

July 2026

What this presentation covers

Implementations

- ▶ Cheap-only block join baseline
- ▶ Expensive-only block join baseline
- ▶ Row-wise cascade
- ▶ Batch-wise cascade

Cascading details

- ▶ Cheap score, confidence, and routing formula
- ▶ Learned confidence threshold from calibration
- ▶ Why batching changed the call/quality trade-off

Evidence

- ▶ Qwen3 0.6B → Qwen3 1.7B, 11 repetitions
- ▶ Precision, recall, F1, runtime, and call counts
- ▶ Threshold sweep with 9 repetitions per point

Conclusion

- ▶ Which variant is strongest in this benchmark
- ▶ What still limits speed and quality
- ▶ Concrete next experiments

Benchmark task

Question

Which movies released in 1998 have reviews expressing an overall negative, critical, or strongly unfavorable opinion of the movie?

Input

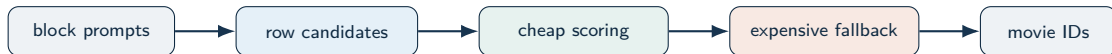
- ▶ 50 movie rows
- ▶ 50 review rows
- ▶ 2,500 raw cross-product pairs
- ▶ 13 true matching movie IDs

Models and repetitions

- ▶ Cheap model: qwen3:0.6b
- ▶ Expensive model: qwen3:1.7b
- ▶ Main comparison: 11 repetitions, mean metrics
- ▶ Threshold sweep: 9 repetitions per threshold

Versions compared

Version	Main idea	LLM use
Cheap block join	Original block nested-loops join with cheap model only	14 cheap block calls
Expensive block join	Original block nested-loops join with expensive model only	14 expensive block calls
Row-wise cascade	Exact-key candidates, cheap score each row, fallback uncertain rows	50 cheap calls + 10 expensive calls
Batch-wise cascade	Batch cheap scoring and batch expensive fallback	7 cheap calls + 3 expensive calls



How cascading worked before the improvements

Original block join

1. Partition movies and reviews into blocks.
2. Ask one LLM call to find all matching rows in each block pair.
3. Parse matching movie IDs from structured output.

Cost is driven by block prompt size and repeated table context.

First row-wise cascade

1. Use exact IDs to create candidate movie-review pairs.
2. Score every candidate with cheap model.
3. Send uncertain candidates to expensive model.

This improves recall but may add overhead: one cheap request per candidate.

Cascade routing formula

For each candidate pair, the cheap model returns one signed score:

$$s_i = \begin{cases} > 0, & \text{cheap model thinks the pair matches,} \\ < 0, & \text{cheap model thinks the pair does not match.} \end{cases}$$

The threshold is applied to the absolute score:

$$|s_i| \geq \tau \iff \text{cheap model is confident enough.}$$

Then the cascade uses only three cases:

$$\text{route}(x_i) = \begin{cases} \text{cheap accept,} & s_i \geq \tau, \\ \text{cheap reject,} & s_i \leq -\tau, \\ \text{expensive fallback,} & -\tau < s_i < \tau. \end{cases}$$

If no threshold is available, every candidate is treated as uncertain and goes to expensive fallback.

Source: `project Trummer/heterogen_v2_3/trummer_join/cascade.py` and `heterogen_v3/trummer_join/cascade.py`.

How cascading was improved

1. Candidate rows

The cascade stops asking the LLM to inspect full movie/review blocks. It first builds exact-key movie-review candidates.

$$50 \times 50 = 2500 \rightarrow 50$$

2. Row-wise cascade

Each candidate receives a cheap-model score. Confident cheap decisions are kept; uncertain rows go to the expensive model.

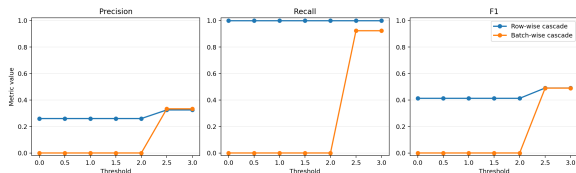
- ▶ High recall.
- ▶ Many cheap calls.

3. Batch-wise cascade

The same cascade logic is kept, but candidates are grouped into batches.

- ▶ 50 cheap calls become 7.
- ▶ Fallback is grouped into 3 expensive calls.

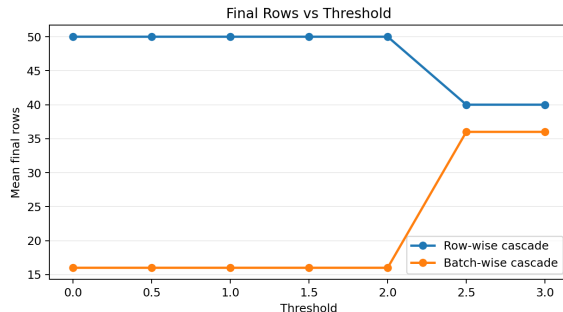
Threshold sweep: what the x-axis means



Interpretation

- ▶ The plot forces $\tau \in \{0, 0.5, 1, 1.5, 2, 2.5, 3\}$ instead of learning it.
- ▶ Smaller τ : cheap model decides more candidates.
- ▶ Larger τ : more candidates are considered uncertain and routed to expensive fallback.
- ▶ In this run, the major transition happens between $\tau = 2.0$ and $\tau = 2.5$ because the cheap scorer often emits bounded scores near ± 2 .

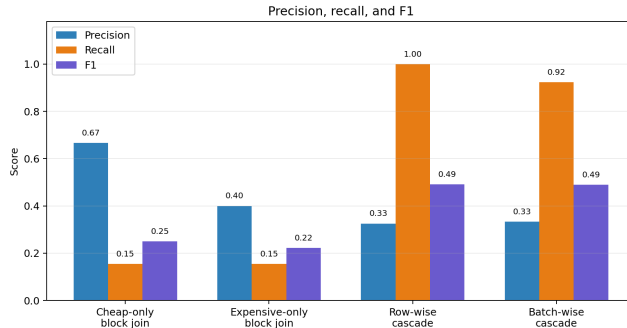
Threshold sweep: final row count



Analysis

- ▶ Row-wise cascade at $\tau \leq 2.0$: cheap accepts all 50 candidates, recall is 1.0 but precision drops to 0.26.
- ▶ Row-wise cascade at $\tau \geq 2.5$: expensive fallback handles the candidates, final rows fall from 50 to 40 and F1 rises to 0.491.
- ▶ Batch-wise cascade at $\tau \leq 2.0$: cheap early rejects 7 candidates; output has 16 false-positive rows and no true positives.
- ▶ Batch-wise cascade at $\tau \geq 2.5$: fallback recovers quality, reaching F1 0.490.

Quality: four-way comparison



- ▶ Cheap-only and expensive-only block joins both miss most true matches.
- ▶ Cascades improve recall and F1; batch-wise cascade nearly matches row-wise F1 with far fewer calls.

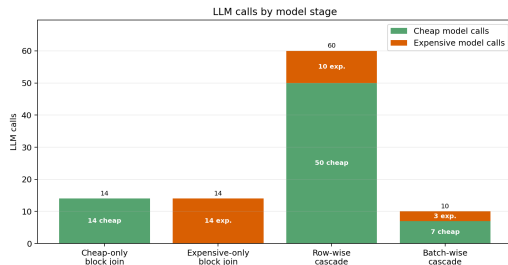
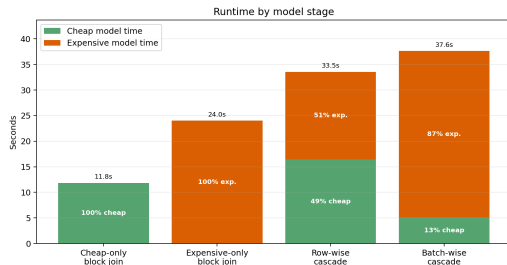
Mean result table

Version	Time (s)	Calls	Final rows	Precision	F1
Cheap block join	11.84	14	3.0	0.667	0.250
Expensive block join	24.00	14	5.0	0.400	0.222
Row-wise cascade	33.56	60	40.0	0.325	0.491
Batch-wise cascade	37.65	10	36.0	0.333	0.490

Main reading

Cascading improves recall and F1 compared with both block-join baselines. Batch-wise cascade is the more call-efficient cascade: it uses 10 total calls instead of 60 while keeping nearly the same F1.

Runtime and call composition



- ▶ Cheap-only block join is fastest, but this speed comes with low recall.
- ▶ Row-wise cascade is call-heavy: 50 cheap calls plus 10 expensive calls.
- ▶ Batch-wise cascade reduces calls from 60 to 10, but runtime is still high because expensive fallback dominates.

Experiment analysis

- ▶ **Cheap-only block join** is fast but misses most true IDs: recall 0.154.
- ▶ **Expensive-only block join** is slower and still low recall in this benchmark, showing that prompt shape matters as much as model size.
- ▶ **Row-wise cascade** maximizes recall but over-produces rows and is slow from one cheap request per candidate.
- ▶ **Batch-wise cascade** keeps almost the same F1 as row-wise cascade while reducing total calls from 60 to 10.
- ▶ **Runtime is not only call count:** batch-wise cascade uses fewer calls, but the expensive fallback batches dominate time.
- ▶ **Best trade-off depends on goal:** cheap-only is fastest; row-wise is best for recall; batch-wise is best for call efficiency.

Suggested further work

More evidence

- ▶ Try the same comparison on more questions, not only the 1998 negative-review task.
- ▶ Run the best variants on Kraken GPU to check whether the speed ranking stays the same.
- ▶ Test a few model pairs, from very small cheap models to stronger expensive models.
- ▶ Report both the average and how much the runs vary.

Better cascades

- ▶ Tune the threshold separately for “accept” and “reject” decisions.
- ▶ Send fewer rows to the expensive model when the cheap model is clearly right.
- ▶ Make batch sizes adapt to the number of uncertain candidates.
- ▶ Avoid expensive fallback when the cheap score is very confident.

Takeaways

1. Heterogeneous semantic joins need more than a smaller cheap model; the candidate set and prompt granularity dominate.
2. Cascading improves recall because uncertain cheap-model decisions can be corrected by the expensive model.
3. The threshold τ is a confidence cutoff on $|s_i|$, not a probability; it controls how much authority the cheap model has.
4. For these four variants, batch-wise cascade is the most call-efficient cascade, while row-wise cascade has the slightly higher F1.